



CODEMIND
AI CODE INTELLIGENCE

CONFIDENTIAL

SECURITY & QUALITY LEDGER

Automated Code Audit & Remediation Report

Deep static analysis and large-language model validation of security vulnerabilities, performance bottlenecks, and structural anomalies.

PROJECT NAME

QuixBugs

AUDIT ID

CM-AUD-02480D

DATE GENERATED

July 15, 2026

OVERALL RATING

90 /100

CONFIDENTIAL — AUDIT BY CODEMIND Generated: July 15, 2026

SECTION 01

Executive Summary

OVERALL SCORE



SECURITY RATING



PERFORMANCE SCORE



MAINTAINABILITY



⚡ AI EXECUTIVE ANALYSIS

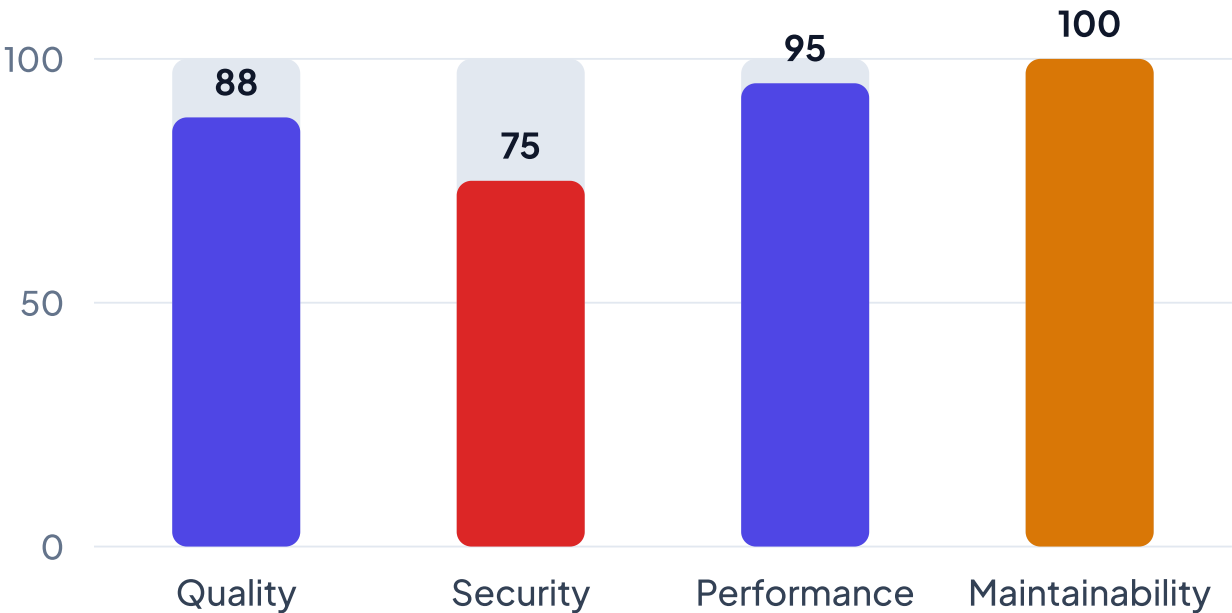
Analysis completed. Scanned 326 files. Found 6 issues. Overall codebase score is 90/100. Critical issues: 1, High priority: 1.

SEVERITY BREAKDOWN



Critical	1
High	1
Medium	3
Low	1

RATING DISTRIBUTIONS



FILES SCANNED

4

LINES ANALYZED

1430

QUALITY HOTSPOTS

TIME ELAPSED

SECTION 02

Detailed Findings & Remediation

#01 JavaDeserialization.java (Line 34)

● CRITICAL

DESCRIPTION

Deserialization of Untrusted Data: The 'getParameters' method deserializes user input using Gson, which can lead to deserialization vulnerabilities.

VULNERABILITY IMPACT

CONFIDENTIALITY & INTEGRITY RISK: An attacker can exploit this vulnerability to execute arbitrary code or access sensitive data.

ORIGINAL CODE

```
1 | Object[] parameters = getParameters(types, args);
```

SUGGESTED CORRECTION

```
1 | Object[] parameters = getParametersSecure(types, args); 400 font-bold">class=400 fon
t-bold">class="text-amber-400">"text-emerald-500">// Implement a secure deserializat
ion method
```

RECOMMENDED REMEDIATION

Use a secure deserialization library or validate user input before deserialization. Suggested Fix: ``java Object[] parameters = getParametersSecure(types, args); // Implement a secure deserialization method ``

#02 JavaDeserialization.java (Line 43)

● HIGH

DESCRIPTION

Insecure Reflection: The 'getParameters' method uses reflection to invoke methods, which can lead to insecure method invocation.

VULNERABILITY IMPACT

INTEGRITY RISK: An attacker can exploit this vulnerability to invoke arbitrary methods.

ORIGINAL CODE

```
1 | Method m = target_class.getDeclaredMethod(methodName, types);
```

SUGGESTED CORRECTION

```
1 | Method m = target_class.getMethod(methodName, types); 400 font-bold">class=400 font-  
bold">class="text-amber-400">"text-emerald-500">// Use getMethod instead of getDecla  
redMethod
```

RECOMMENDED REMEDIATION

Use a secure reflection library or validate method invocation. Suggested Fix: ``java Method m = target\_class.getMethod(methodName, types); // Use getMethod instead of getDeclaredMethod ``

#03 JavaDeserialization.java (Line 51)

• MEDIUM

DESCRIPTION

Potential NullPointerException: The 'getParameters' method does not check for null values before invoking methods.

VULNERABILITY IMPACT

AVAILABILITY RISK: A NullPointerException can occur if a null value is passed to the method.

ORIGINAL CODE

```
1 | parameters[i] = gsonArguments.fromJson(args[i + 1], (Class)type);
```

SUGGESTED CORRECTION

```
1 | 400 font-bold">if (args[i + 1] ≠ null) { parameters[i] = gsonArguments.fromJson(arg  
  | s[i + 1], (Class)type); }
```

RECOMMENDED REMEDIATION

Add null checks before invoking methods. Suggested Fix: ``java if (args[i + 1] != null) { parameters[i] = gsonArguments.fromJson(args[i + 1], (Class)type); } ``

#04 correct_java_programs / BUCKETSORT.java (Line 15)

• LOW

DESCRIPTION

Potential IndexOutOfBoundsException: The 'bucketsort' method does not check if the input array is empty before accessing its elements.

VULNERABILITY IMPACT

AVAILABILITY RISK: An IndexOutOfBoundsException can occur if the input array is empty.

ORIGINAL CODE

```
1 | counts.set(x,counts.get(x)+1);
```

SUGGESTED CORRECTION

```
1 | 400 font-bold">if (arr.isEmpty()) { 400 font-bold">return 400 font-bold">new ArrayLi  
st<>(); } 400 font-bold">class=400 font-bold">class="text-amber-400">"text-emerald-5  
00">// Add a check 400 font-bold">for an empty input array
```

RECOMMENDED REMEDIATION

Add a check for an empty input array. Suggested Fix: ``java if (arr.isEmpty()){ return new ArrayList<>(); }// Add a check for an empty input array ``

#05  correct_java_programs / DEPTH_FIRST_SEARCH.java (Line 20)

• MEDIUM

DESCRIPTION

Inefficient Recursive Method: The 'depth_first_search' method uses a recursive approach, which can lead to stack overflow for large inputs.

VULNERABILITY IMPACT

AVAILABILITY RISK: A stack overflow can occur for large inputs.

ORIGINAL CODE

```
1 | 400 font-bold">class Search { boolean search(Node node) {
```

SUGGESTED CORRECTION

```
1 | boolean search(Node node) { 400 font-bold">class=400 font-bold">class="text-amber-400">"text-emerald-500"> // Use an iterative approach instead of recursion
```

RECOMMENDED REMEDIATION

Use an iterative approach instead of recursion. Suggested Fix: ``java boolean search(Node node){ // Use an iterative approach instead of recursion ``

#06  correct_java_programs / DETECT_CYCLE.java (Line 15)

• MEDIUM

DESCRIPTION

Potential Infinite Loop: The 'detect_cycle' method does not check for a null node before entering the loop.

VULNERABILITY IMPACT

AVAILABILITY RISK: An infinite loop can occur if the input node is null.

ORIGINAL CODE

```
1 400 font-bold">while (true) {
```

SUGGESTED CORRECTION

```
1 400 font-bold">if (node == null) { 400 font-bold">return false; } 400 font-bold">cla  
ss=400 font-bold">class="text-amber-400">"text-emerald-500">// Add a check 400 font-  
bold">for a null node
```

RECOMMENDED REMEDIATION

Add a check for a null node before entering the loop. Suggested Fix: ``java if (node == null) { return false; } // Add a check for a null node ``

SECTION 03

Summary Recommendations



Security Pipeline

- Use a secure deserialization library or validate user input before deserialization. Suggested Fix: ```java Object[] parameters = getParametersSecure(types, args); // Implement a secure deserialization method ```
- Use a secure reflection library or validate method invocation. Suggested Fix: ```java Method m = target_class.getMethod(methodName, types); // Use getMethod instead of getDeclaredMethod ```



Performance & Scale

- Add a check for a null node before entering the loop. Suggested Fix: ```java if (node == null) { return false; } // Add a check for a null node ```



Code Quality & Structure

- Add null checks before invoking methods. Suggested Fix: ```java if (args[i + 1] != null) { parameters[i] = gsonArguments.fromJson(args[i + 1], (Class)type); } ```
- Use an iterative approach instead of recursion. Suggested Fix: ```java boolean search(Node node) { // Use an iterative approach instead of recursion ```
- Add a check for a null node before entering the loop. Suggested Fix: ```java if (node == null) { return false; } // Add a check for a null node ```



Industry Best Practices

- Add a check for an empty input array. Suggested Fix: ```java if (arr.isEmpty()) { return new ArrayList<>(); } // Add a check for an empty input array ```